Game Week Project Guide

Background

Most of you have never built a game before. That's exactly the point.

This week isn't about becoming game developers. It's about proving you can enter completely unfamiliar territory and ship polished, production-quality software using AI as your learning engine.

In the real world, you'll face situations where your boss assigns you to a project using technologies you've never touched. Maybe it's a legacy system in COBOL. Maybe it's a cutting-edge framework that launched last month. Maybe it's an industry-specific tool that nobody on your team has experience with. The question becomes: Can you get up to speed fast enough to contribute meaningfully?

This project tests your ability to:

- Rapidly acquire expertise in unfamiliar domains using AI
- Make informed technical decisions without prior experience
- Build a methodology for AI-accelerated learning that you can apply anywhere
- Deliver polished results despite starting from zero

We're using game development as the vehicle because:

1. It's likely unfamiliar territory for most of you
2. Games demand technical excellence - performance issues are immediately visible
3. The quality bar is clear - it either feels good to play or it doesn't
4. Multiplayer requirements add genuine technical complexity

By the end of this week, you'll have developed a repeatable process for using AI to master new technologies and deliver professional results in record time.

Core Challenge

You will develop a multiplayer game using technologies you have never worked with before. This project tests your ability to:

- Research and learn new technologies rapidly using AI
- Make informed technical decisions under time constraints
- Build production-quality software in unfamiliar territory
- Demonstrate that AI-augmented developers can match or exceed traditional development timelines

Technical Requirements

Game Specifications

- **Multiplayer Support:** Real-time interaction between multiple players
- **Performance:** Low latency, high performance gameplay with no lag
- **Platform:** Your choice - mobile, web, or desktop application
- **Complexity:** Must include levels or character progression - players need a sense of advancement
- **Engagement:** Fun, interesting gameplay with clear objectives or storyline

Development Pathways

Choose one of these pathways based on your target platform. Each pathway has been selected for having excellent documentation, active communities, and AI-friendly learning resources:

Path 1: Desktop Game Development with Godot

- **Engine:** Godot
- **Language:** GDScript (Python-like syntax)
- **Networking:** Godot's built-in multiplayer API
- **Why this path:** Godot has exceptional documentation, thousands of tutorials, and GDScript is designed to be beginner-friendly. The engine handles many complex tasks for you while still teaching core game development concepts.

Path 2: Browser-Based Game

- **Framework:** Phaser 3 (2D) or Three.js (3D)
- **Language:** JavaScript
- **Networking:** Socket.io for real-time multiplayer
- **Why this path:** Both frameworks have extensive documentation. Phaser 3 is perfect for 2D games with built-in physics and animations. Three.js opens up 3D possibilities if you want an extra challenge. Socket.io simplifies multiplayer implementation.

Path 3: Unity for Cross-Platform

- **Engine:** Unity
- **Language:** C#
- **Networking:** Unity Netcode for GameObjects or Mirror Networking
- **Platform:** Desktop or WebGL build

- **Why this path:** Unity has the largest game development community, countless tutorials, and comprehensive documentation. C# is well-structured and extensively covered by AI training data.

Development Framework

Day 1-2: Research and Learning Phase

Objective: Become competent in your chosen tech stack

1. **Technology Selection**
    - Analyze game requirements against available technologies
    - Use AI to compare pros/cons of different tech stacks
    - Make an informed decision based on project constraints
2. **Create Your Learning Path**
    - Use AI to generate a customized curriculum for your chosen stack
    - Focus on essential concepts needed for your game
    - Build small proof-of-concept demos to validate understanding
3. **Architecture Planning**
    - Design your game architecture with AI assistance
    - Plan networking infrastructure for multiplayer support
    - Identify potential performance bottlenecks early

Day 3-5: Core Development Phase

Objective: Build the fundamental game systems

1. **Game Mechanics Implementation**
    - Start with single-player mechanics
    - Use AI to debug and optimize unfamiliar code patterns
    - Implement core gameplay loop
2. **Multiplayer Integration**
    - Add networking layer
    - Implement state synchronization
    - Handle player connections/disconnections

3. **Performance Optimization**
    - Profile and identify bottlenecks
    - Use AI to suggest optimization strategies
    - Ensure smooth gameplay with multiple concurrent players

Day 6-7: Polish and Testing Phase

Objective: Create a polished, playable experience

1. **Gameplay Enhancement**
    - Add progression systems or storyline elements
    - Implement UI/UX improvements
    - Balance game mechanics based on testing
2. **Stress Testing**
    - Test with maximum expected concurrent players
    - Verify low-latency performance
    - Fix any remaining bugs or issues
3. **Documentation**
    - Create setup and deployment instructions
    - Document your learning process and AI utilization
    - Prepare demonstration materials

Evaluation Criteria

Your project will be assessed on:

1. **Technical Achievement**
    - Successfully implementing multiplayer functionality
    - Meeting performance requirements
    - Code quality despite unfamiliar tech stack
2. **Learning Velocity & Methodology**
    - How quickly you became productive in new technologies
    - Documentation of your learning tools and resources
    - Specific AI prompts and techniques that accelerated learning
    - Learning pathway decisions and pivots
3. **Game Quality**
    - Fun factor and engagement

    - Implementation of levels or progression system
    - Polish and attention to detail
4. **AI Utilization**
    - Strategic use of AI tools for learning
    - Problem-solving efficiency with AI assistance
    - Innovation in applying AI to development challenges
    - Quality of prompts and interactions with AI

Deliverables

1. **Working Game**
    - Deployed and accessible for testing
    - Supporting multiple concurrent players
    - Meeting all technical requirements
2. **GitHub Repository**
    - Architecture overview
    - Setup and deployment guide
    - Key technical decisions and rationale
3. **Brainlift**
    - Daily progress updates
    - AI prompts and interactions that accelerated learning
    - Challenges faced and solutions found
4. **Demo Video**
    - 5-minute gameplay demonstration
    - Technical walkthrough of key features
    - Reflection on AI-augmented development process

Success Metrics

You will have succeeded if you can demonstrate:

- A fully functional multiplayer game in a previously unknown tech stack
- Development velocity comparable to or exceeding traditional methods
- Effective AI utilization throughout the learning and building process
- A fun, engaging game that showcases technical competence

Final Notes

This project simulates real-world scenarios where developers must quickly adapt to new technologies. Your ability to leverage AI for rapid learning and development is the key differentiator. Focus on smart decision-making, efficient learning paths, and pragmatic implementation choices.

Remember: The goal isn't perfection in 7 days—it's demonstrating that AI-augmented developers can achieve in one week what might traditionally take much longer when facing unfamiliar technology stacks.